Petpooja AI Copilot

ML Model Implementation & Training Guide

A comprehensive technical guide covering four proposed ML models for the Petpooja restaurant intelligence platform:

Demand Forecasting · NLP Intent Parsing
Collaborative Filtering · Anomaly Detection

Each section includes: architecture overview, data requirements, step-by-step training code, evaluation metrics, and integration strategy.

March 6, 2026

Contents

1 Demand Forecasting (Time-Series Prediction) 2
- 1.1 How It Works . . . 2
- 1.2 Data Requirements . . . 2
  - 1.2.1 Minimum Data . . . 2
  - 1.2.2 Data Schema (from `orders.json`) . . . 2
  - 1.2.3 Feature Engineering . . . 2
- 1.3 Step-by-Step Training . . . 3
  - 1.3.1 Step 1: Install Dependencies . . . 3
  - 1.3.2 Step 2: Prepare Training Data . . . 3
  - 1.3.3 Step 3: Train a Prophet Model Per Item . . . 3
  - 1.3.4 Step 4: Generate Predictions . . . 4
  - 1.3.5 Step 5: Connect to Inventory . . . 4
- 1.4 Integration with Petpooja . . . 4
- 1.5 Evaluation Metrics . . . 4

2 NLP Intent Parser (LLM-Based Order Understanding) 5
- 2.1 Why LLM Over Regex + Fuzzy Match? . . . 5
- 2.2 Implementation Strategy . . . 5
  - 2.2.1 Option A: Local Model (Google Gemma 2B) . . . 5
  - 2.2.2 Option B: Cloud API (OpenAI GPT-4o-mini) . . . 6
- 2.3 Fine-Tuning (Optional — Advanced) . . . 7
  - 2.3.1 Step 1: Collect Training Data . . . 7
  - 2.3.2 Step 2: Fine-Tune with QLoRA . . . 7
- 2.4 Evaluation Metrics . . . 7

3 Collaborative Filtering (Personalized Upsell) 8
- 3.1 How It Works . . . 8
  - 3.1.1 User–Item Matrix Example . . . 8
- 3.2 Data Requirements . . . 8
- 3.3 Step-by-Step Training . . . 8
  - 3.3.1 Step 1: Install Dependencies . . . 8
  - 3.3.2 Step 2: Prepare Interaction Data . . . 8
  - 3.3.3 Step 3: Train SVD Model . . . 9
  - 3.3.4 Step 4: Generate Recommendations . . . 9
  - 3.3.5 Step 5: Cold-Start Fallback . . . 10
- 3.4 Integration with Petpooja . . . 10
- 3.5 Evaluation Metrics . . . 10

4 Anomaly Detection (Revenue & Inventory) 11
- 4.1 Types of Anomalies Detected . . . 11
- 4.2 Data Requirements . . . 11
- 4.3 Step-by-Step Training . . . 11
  - 4.3.1 Step 1: Install Dependencies . . . 11
  - 4.3.2 Step 2: Statistical Anomaly Detection (Z-Score) . . . 11
  - 4.3.3 Step 3: Isolation Forest (Multi-Dimensional) . . . 12

4.3.4 Step 4: Real-Time Scoring 12
4.3.5 Step 5: Root Cause Classification 13
4.4 Integration with Petpooja 13
4.5 Evaluation Metrics 14

5 Summary & Technology Stack 15
5.1 Model Comparison 15
5.2 Full Technology Stack 15
5.3 Implementation Roadmap 15

1. Demand Forecasting (Time-Series Prediction)

Model Overview

Goal	Predict how many units of each menu item will sell in the next N days
Algorithm	Facebook Prophet (recommended) / LSTM / ARIMA
Library	`prophet`, `pandas`, `numpy`
Priority	**HIGH** — connects Orders → Recipes → Inventory → Predictions

1.1 How It Works

Time-series forecasting models analyze historical order patterns to predict future demand. The model learns:

- **Weekly seasonality** — weekday vs. weekend traffic patterns
- **Daily seasonality** — lunch rush vs. dinner rush vs. off-hours
- **Trend** — overall growth or decline in item popularity
- **External regressors** — holidays, weather, local events (optional)

The output directly feeds into the inventory system: *"You will need 15 kg chicken and 200 burger buns by Friday."*

1.2 Data Requirements

1.2.1 Minimum Data

At least **30 days** of timestamped order history. Accuracy improves significantly with 6+ months of data.

1.2.2 Data Schema (from `orders.json`)

```
{
  "order_id": "ORD-20260301-001",
  "timestamp": "2026-03-01T12:34:00",
  "items": [
    { "item_id": "M001", "name": "Classic Veg Burger", "qty":
       2 },
    { "item_id": "M019", "name": "French Fries", "qty": 1 }
  ]
}
```

1.2.3 Feature Engineering

Derive the following features from the raw timestamp:

- `day_of_week` (0=Mon, 6=Sun) — captures weekly patterns
- `hour_of_day` (0–23) — captures meal-time peaks
- `is_weekend` (boolean) — weekend traffic is typically 30–40% higher

- `is_holiday` (boolean) — from a public holiday calendar
- `rolling_avg_7d` — 7-day rolling average quantity per item
- `weather_temp` (optional) — from a weather API; affects beverage sales

1.3 Step-by-Step Training

1.3.1 Step 1: Install Dependencies

```
pip install prophet pandas numpy scikit-learn
```

1.3.2 Step 2: Prepare Training Data

```
import pandas as pd
import json

# Load historical orders
with open("data/orders.json") as f:
    orders = json.load(f)

# Flatten to daily item counts
rows = []
for order in orders:
    dt = pd.to_datetime(order["timestamp"]).date()
    for item in order["items"]:
        rows.append({
            "ds": dt,
            "item_id": item["item_id"],
            "y": item["qty"]
        })

df = pd.DataFrame(rows)
daily = df.groupby(["ds", "item_id"])["y"].sum().reset_index()
```

1.3.3 Step 3: Train a Prophet Model Per Item

```
from prophet import Prophet
import pickle

models = {}
for item_id in daily["item_id"].unique():
    item_data = daily[daily["item_id"] == item_id][["ds", "y"]]

    model = Prophet(
        daily_seasonality=True,
        weekly_seasonality=True,
        yearly_seasonality=False,   # need 2+ years of data
        changepoint_prior_scale=0.05
    )
    model.fit(item_data)
```

```
    models[item_id] = model

# Save all trained models
with open("models/demand_models.pkl", "wb") as f:
    pickle.dump(models, f)
```

1.3.4 Step 4: Generate Predictions

```
# Predict next 7 days for each item
for item_id, model in models.items():
    future = model.make_future_dataframe(periods=7)
    forecast = model.predict(future)

    pred = forecast.tail(7)[["ds", "yhat", "yhat_lower",
        "yhat_upper"]]
    print(f"{item_id}: next 7 days forecast")
    print(pred.to_string(index=False))
```

1.3.5 Step 5: Connect to Inventory

```
# For each predicted item demand, calculate ingredient needs
recipes = load_recipes()        # from recipes.json
inventory = load_inventory()    # from inventory.json

for item_id, predicted_qty in predictions.items():
    recipe = recipes.get(item_id, {})
    for ing in recipe.get("ingredients", []):
        needed = ing["qty"] * predicted_qty
        current = get_stock(ing["ingredient_id"])
        if current < needed:
            shortfall = needed - current
            alert(f"Order {shortfall} {ing['unit']} of
                {ing['name']}")
```

1.4 Integration with Petpooja

- Create API endpoint: `GET /forecast/{item_id}?days=7`
- Run daily cron job to retrain models at 2:00 AM
- Display predictions on the Dashboard with confidence bands (chart)
- Auto-generate purchase order suggestions when predicted demand exceeds stock

1.5 Evaluation Metrics

Metric	Description	Target
MAE	Mean Absolute Error per item/day	< 3 units
MAPE	Mean Absolute Percentage Error	$< 15\%$
Coverage	% of actuals within prediction interval	$> 90\%$

2. NLP Intent Parser (LLM-Based Order Understanding)

Model Overview

Goal	Replace regex + fuzzy-match parsing with an LLM that understands complex natural-language orders
Algorithm	Large Language Model (prompt engineering / fine-tuning)
Library	`transformers` + `torch` (local) **or** `openai` SDK (cloud)
Priority	**HIGH** — dramatically improves voice order accuracy

2.1 Why LLM Over Regex + Fuzzy Match?

The current system uses RapidFuzz for fuzzy string matching, which works well for simple orders but fails on:

- **Corrections:** "Give me 2 burgers, actually make it 3"
- **References:** "Same thing as last order but no onions"
- **Multi-item modifiers:** "One margherita and one pepperoni, extra cheese on both"
- **Quantities in words:** "A couple of cold coffees and half a dozen samosas"
- **Hinglish/slang:** "Ek paneer wala pizza aur do chai de do"

An LLM understands context, intent, and corrections naturally.

2.2 Implementation Strategy

2.2.1 Option A: Local Model (Google Gemma 2B)

This runs entirely on your machine — no API cost, no data leaving your server.

```
pip install transformers torch accelerate
```

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import json

# Load model (first run downloads ~5 GB)
model_name = "google/gemma-2b-it"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)

def parse_order_llm(transcription: str, menu_items: list) ->
    dict:
    menu_str = "\n".join(
        f"- {m['item_id']}: {m['name']} (Rs
            {m['selling_price']})"
        for m in menu_items
    )
```

```
    prompt = f"""You are a restaurant order parser.
Given the spoken order and menu, extract items as JSON.

MENU:
{menu_str}

CUSTOMER ORDER: "{transcription}"

Respond ONLY with valid JSON:
{{"items": [{{"item_id": "...", "name": "...", "qty": N,
              "modifiers": [...], "price": N}}],
  "total": N}}"""

    inputs = tokenizer(prompt, return_tensors="pt")
    outputs = model.generate(**inputs, max_new_tokens=512,
                             temperature=0.1)
    response = tokenizer.decode(outputs[0],
       skip_special_tokens=True)

    # Extract JSON from response
    json_start = response.rfind("{")
    json_end = response.rfind("}") + 1
    return json.loads(response[json_start:json_end])
```

2.2.2 Option B: Cloud API (OpenAI GPT-4o-mini)

Lower latency (< 1s), higher accuracy, but costs $0.15 per 1M input tokens.

```
import openai, json

client = openai.OpenAI(api_key="sk-...")

def parse_order_api(transcription: str, menu_items: list) ->
   dict:
    menu_str = "\n".join(
        f"- {m['item_id']}: {m['name']} (Rs
           {m['selling_price']})"
        for m in menu_items
    )

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        response_format={"type": "json_object"},
        messages=[
            {"role": "system",
             "content": f"You parse restaurant
                orders.\nMENU:\n{menu_str}"},
            {"role": "user", "content": transcription}
        ],
        temperature=0.1
    )
```

```
    return json.loads(response.choices[0].message.content)
```

2.3 Fine-Tuning (Optional — Advanced)

For the highest accuracy, fine-tune a small model on your own data.

2.3.1 Step 1: Collect Training Data

Create a JSONL file with 100–500 labeled examples:

```
{"input": "2 paneer burgers extra cheese and a cold coffee",
 "output": {"items": [
   {"item_id": "M002", "name": "Paneer Burger", "qty": 2,
    "modifiers": ["extra cheese"]},
   {"item_id": "M027", "name": "Cold Coffee", "qty": 1,
    "modifiers": []}
 ]}}
```

2.3.2 Step 2: Fine-Tune with QLoRA

```
pip install peft trl datasets bitsandbytes
```

QLoRA allows fine-tuning a 2B parameter model on a single GPU (8 GB VRAM) by quantizing the base model to 4-bit and training only small adapter layers.

- Training data: 100–500 labeled examples
- Training time: ∼30 minutes on a single GPU
- Expected result: 95%+ accuracy on order parsing

2.4 Evaluation Metrics

Metric	Description	Target
Exact Match Rate	% of orders with 100% correct items	$> 90\%$
Item-Level F1	Precision/recall for item extraction	> 0.95
Modifier Accuracy	% of modifiers correctly captured	$> 85\%$
Latency	End-to-end response time	< 2s (local), < 1s (API)

3. Collaborative Filtering (Personalized Upsell)

Model Overview

Goal	Learn customer ordering patterns to suggest personalized upsells
Algorithm	SVD (Matrix Factorization) / Neural Collaborative Filtering
Library	`scikit-surprise` (SVD) or `tensorflow` (Neural CF)
Priority	MEDIUM — requires customer identity tracking

3.1 How It Works

Collaborative Filtering (CF) builds a **user–item interaction matrix** from order history. It discovers that "customers who ordered Paneer Pizza also loved Garlic Bread" and generates personalized recommendations.

1. Build matrix: rows = customers, columns = menu items, values = order frequency
2. Factorize the matrix into latent factors using SVD
3. For a given customer, predict scores for un-ordered items
4. Return top-K items with highest predicted scores

3.1.1 User–Item Matrix Example

	M001	M003	M009	M019	M027
Customer_001	5	0	3	4	2
Customer_002	0	7	1	6	0
Customer_003	3	2	0	1	5

Values = number of times customer ordered that item.

3.2 Data Requirements

- Customer identification: phone number, loyalty ID, or table session
- Minimum: **500 unique customers** with **3+ orders each**
- More data = better recommendations (cold-start is the main challenge)

3.3 Step-by-Step Training

3.3.1 Step 1: Install Dependencies

```
pip install scikit-surprise pandas numpy
```

3.3.2 Step 2: Prepare Interaction Data

```
import pandas as pd
from surprise import Dataset, Reader, SVD
from surprise.model_selection import cross_validate
```

```
# Build user-item-rating dataframe from orders
rows = []
for order in orders:
    cust_id = order.get("customer_id", "anonymous")
    for item in order["items"]:
        rows.append({
            "user": cust_id,
            "item": item["item_id"],
            "rating": item["qty"]    # frequency as implicit
               rating
        })

df = pd.DataFrame(rows)
df = df.groupby(["user", "item"])["rating"].sum().reset_index()
df["rating"] = df["rating"].clip(1, 5)  # normalize to 1-5
   scale
```

3.3.3 Step 3: Train SVD Model

```
reader = Reader(rating_scale=(1, 5))
data = Dataset.load_from_df(df[["user", "item", "rating"]],
   reader)

# Train SVD
algo = SVD(n_factors=50, n_epochs=30, lr_all=0.005,
   reg_all=0.02)
trainset = data.build_full_trainset()
algo.fit(trainset)

# Cross-validate
results = cross_validate(algo, data, measures=["RMSE", "MAE"],
   cv=5)
print(f"RMSE: {results['test_rmse'].mean():.3f}")
print(f"MAE:  {results['test_mae'].mean():.3f}")
```

3.3.4 Step 4: Generate Recommendations

```
def get_recommendations(customer_id, menu_items, top_k=5):
    """Get top-K item recommendations for a customer."""
    ordered = set(df[df["user"] == customer_id]["item"])
    candidates = [m["item_id"] for m in menu_items
                  if m["item_id"] not in ordered]

    predictions = [
        (iid, algo.predict(customer_id, iid).est)
        for iid in candidates
    ]
    predictions.sort(key=lambda x: x[1], reverse=True)
```

```
    return predictions[:top_k]
```

3.3.5 Step 5: Cold-Start Fallback

```
def get_popular_items(top_k=5):
    """Fallback for new customers with no order history."""
    popularity = df.groupby("item")["rating"].sum()
    return popularity.nlargest(top_k).index.tolist()
```

3.4 Integration with Petpooja

- Create endpoint: `GET /recommend/{customer_id}?top_k=5`
- Add `customer_id` field to the order flow (phone number or loyalty card)
- Replace generic Apriori upsells with personalized CF recommendations
- Fall back to popularity-based suggestions for new customers
- Weekly retraining via cron job with latest order data

3.5 Evaluation Metrics

Metric	Description	Target
RMSE	Root Mean Square Error on held-out data	< 1.0
Hit Rate@5	% of test orders in top-5 recommendations	$> 30\%$
NDCG@5	Normalized Discounted Cumulative Gain	> 0.4
Coverage	% of catalog items ever recommended	$> 60\%$

4. Anomaly Detection (Revenue & Inventory)

Model Overview

Goal	Detect unusual patterns in revenue, orders, and inventory in real-time
Algorithm	Z-Score (statistical) + Isolation Forest (ML)
Library	`scikit-learn`, `numpy`, `pandas`
Priority	MEDIUM — critical for loss prevention

4.1 Types of Anomalies Detected

- **Revenue:** Sudden revenue drop > 30% compared to same day last week
- **Orders:** Unusually high void/cancel rate (potential fraud)
- **Inventory:** Ingredients depleting faster than order volume justifies (theft/waste)
- **Timing:** Orders placed outside business hours (system misuse)
- **Pricing:** Unauthorized discount patterns

4.2 Data Requirements

- Hourly revenue totals — at least 14 days (30+ recommended)
- Order-level data with timestamps, void status, discount flags
- Inventory movement logs (depletion events with timestamps)

4.3 Step-by-Step Training

4.3.1 Step 1: Install Dependencies

```
pip install scikit-learn pandas numpy
```

4.3.2 Step 2: Statistical Anomaly Detection (Z-Score)

Simple but effective for single-metric monitoring (e.g., daily revenue):

```
import numpy as np
import pandas as pd

def detect_revenue_anomalies(daily_revenue, threshold=2.5):
    """Flag days where revenue deviates > 2.5 std from mean."""
    mean = daily_revenue.mean()
    std = daily_revenue.std()
    z_scores = (daily_revenue - mean) / std

    anomalies = daily_revenue[abs(z_scores) > threshold]
    return [
        {
            "date": str(date),
            "revenue": float(val),
```

```
            "z_score": float(z),
            "type": "low" if z < 0 else "high"
        }
        for date, val, z in zip(
            anomalies.index, anomalies.values,
            z_scores[abs(z_scores) > threshold]
        )
    ]
```

4.3.3 Step 3: Isolation Forest (Multi-Dimensional)

For detecting complex anomalies across multiple features simultaneously:

```
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler

# Feature matrix: each row = 1 hour of operations
# Columns: [revenue, order_count, avg_order_value,
#            void_rate, discount_rate, inventory_depletion]
features = build_feature_matrix(orders, inventory_logs)

# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(features)

# Train Isolation Forest
model = IsolationForest(
    n_estimators=200,
    contamination=0.05,   # expect ~5% anomalies
    max_features=0.8,
    random_state=42
)
model.fit(X_scaled)

# Predict: -1 = anomaly, 1 = normal
predictions = model.predict(X_scaled)
scores = model.decision_function(X_scaled)
print(f"Anomalies found: {(predictions == -1).sum()}")
```

4.3.4 Step 4: Real-Time Scoring

```
import pickle

# Save model + scaler for production use
with open("models/anomaly_model.pkl", "wb") as f:
    pickle.dump({"model": model, "scaler": scaler}, f)

def score_current_hour(current_features):
    """Score the current hour's data for anomalies."""
    with open("models/anomaly_model.pkl", "rb") as f:
```

```
        bundle = pickle.load(f)

    X = bundle["scaler"].transform([current_features])
    is_anomaly = bundle["model"].predict(X)[0] == -1
    score = bundle["model"].decision_function(X)[0]

    if is_anomaly:
        return {
            "alert": True,
            "severity": "HIGH" if score < -0.3 else "MEDIUM",
            "score": float(score),
            "message": classify_anomaly(current_features)
        }
    return {"alert": False}
```

4.3.5 Step 5: Root Cause Classification

```
def classify_anomaly(features):
    """Determine the most likely root cause."""
    revenue, orders, aov, voids, discounts, depletion =
       features

    if voids > historical_void_mean * 2:
        return "High void rate - possible fraud"
    if depletion > expected_depletion * 1.5:
        return "Excessive inventory depletion - check for
           waste/theft"
    if revenue < historical_revenue_mean * 0.6:
        return "Significant revenue drop - investigate
           operations"
    if discounts > historical_discount_mean * 2:
        return "Unusual discount pattern - verify
           authorization"
    return "General anomaly - manual review needed"
```

4.4 Integration with Petpooja

- Create endpoint: `GET /alerts/anomalies?hours=24`
- Run scoring every hour via APScheduler background task
- Display alerts on Dashboard with severity badges (HIGH / MEDIUM / LOW)
- Send push notifications for HIGH severity anomalies
- Weekly model retraining on a rolling 60-day window

4.5 Evaluation Metrics

Metric	Description	Target
Precision	% of flagged anomalies that are real	$> 80\%$
Recall	% of actual anomalies caught	$> 90\%$
False Positive Rate	Avoid alert fatigue	$< 5\%$
Time to Detect	Anomaly flagged within N hours	< 1 hour

5. Summary & Technology Stack

5.1 Model Comparison

Model	Algorithm	Library	Min Data	Priority
Demand Forecasting	Prophet / LSTM	`prophet`	30 days orders	HIGH
NLP Intent Parser	LLM (Gemma/GPT)	`transformers`	Menu + prompt	HIGH
Collaborative Filter	SVD / Neural CF	`surprise`	500 customers	MEDIUM
Anomaly Detection	Isolation Forest	`scikit-learn`	14 days ops data	MEDIUM

5.2 Full Technology Stack

- **Core:** Python 3.12, FastAPI, pandas, numpy
- **Speech:** OpenAI Whisper (already implemented)
- **NLP:** `transformers` + `torch` (local) or `openai` SDK (cloud)
- **Forecasting:** `prophet` (Facebook) or `tensorflow`
- **Recommendations:** `scikit-surprise`
- **Anomaly Detection:** `scikit-learn` (Isolation Forest)
- **Scheduling:** APScheduler or Celery for periodic retraining
- **Model Storage:** Pickle files or MLflow for versioning
- **Hardware:** GPU recommended for NLP model (NVIDIA T4 minimum)

5.3 Implementation Roadmap

1. **Phase 1 (Weeks 1–2):** Implement Demand Forecasting with Prophet. Connect predictions to inventory alerts on the Dashboard.
2. **Phase 2 (Weeks 3–4):** Integrate NLP Intent Parser. Start with cloud API (GPT-4o-mini) for quick results, migrate to local Gemma model later.
3. **Phase 3 (Weeks 5–8):** Deploy Collaborative Filtering once customer identity tracking is in place. Requires loyalty or phone-number based identification.
4. **Phase 4 (Weeks 9–12):** Add Anomaly Detection. Start with Z-Score for revenue monitoring, then add Isolation Forest for multi-dimensional analysis.